

TD - Day & Night - Partie 2

Flutter



Objectifs.....	2
Etape 2.01 : Store de données.....	3

Objectifs

- Limiter le couplage entre widgets.
- Favoriser la communication indirecte entre widgets à travers la diffusion d'événements au sein de l'application (émetteur / abonné).
- Mettre en place un store de données avec, au choix :
 - **Provider** (<https://pub.dev/packages/provider>) ou
 - **FlutterRiverpod** (https://pub.dev/packages/flutter_riverpod).

Etape 2.01 : Store de données

- On part du principe que vous employez le package **Provider**. Si vous avez sélectionné le package **Riverpod**, adaptez les consignes ci-dessous en cohérence.
- Consultez la documentation du package et le tutoriel suivant : <https://docs.flutter.dev/data-and-backend/state-mgmt/simple>
- Programmez une classe **ThemeProvider** héritant de la classe **ChangeNotifier**.
- Gérez le thème sélectionné par l'utilisateur au sein de la classe **ThemeProvider** via un attribut **theme** et employez la méthode **notifyListeners** héritée afin de diffuser un événement auprès des widgets concernés en cas de changement de la valeur de l'attribut **theme**.
- Imbriquez les widgets concernés par l'information partagée par la classe **ThemeProvider** dans des widgets de type **Consumer**.
- Supprimez les références entre les widgets **MainApp** et **DayNight** mises en place dans la première partie du TD (référence vers la méthode *callback* communiquée par le widget *parent* au widget *enfant*).
- En employant un widget **Consumer** abonné à la classe **ThemeProvider** pour mettre à jour dynamiquement son thème, la classe **MainApp** ne gère plus d'état, elle n'a donc plus besoin d'hériter de la classe **StatefulWidget**, convertissez-la en **StatelessWidget** et procédez aux adaptations nécessaires.

Fin